SRS - AUA Rentals Vehicle Rental System

Oct 25, 2026 · @Ashar

AUA Rentals is a console application in C++ where owners list vehicles, customers rent them by the kilometre, and an admin oversees users, vehicles and revenue.

1. Introduction

This SRS specifies what AUA Rentals does today, written by reverse-engineering the program and following the standard requirements engineering stages (elicit, analyse, specify, validate, manage).

1.1 Purpose

To give developers, testers and the course instructor one agreed description of the system's behaviour, so the code can be verified against it.

1.2 Scope

AUA Rentals is a single-machine, text-menu program. It lets owners register and list up to 3 vehicles each, lets customers register, rent one vehicle at a time and return it for a per-kilometre charge, and lets an admin view users, vehicles and total revenue. Data is stored in plain text files. Online payment, GPS tracking, booking by date and a graphical interface are out of scope.

1.3 Definitions

Term	Meaning
Customer	A registered person who rents vehicles
Owner	A registered person who lists vehicles for rent
Admin	The single built-in manager account
Mileage	The vehicle's odometer reading in km
Rent per km	The charge in RS for each km driven during a rental
Rental cost	(return mileage - mileage at pickup) x rent per km

Term	Meaning
Revenue	Sum of all rental costs collected since the data files were created

1.4 References

- IEEE Std 830-1998, Recommended Practice for Software Requirements Specifications
- Source file `car_rental.cpp` (C++11, standard library only)

2. Overall description

The product is a standalone program with three portals behind one main menu, all sharing one in-memory data set that is saved to text files.

2.1 Product perspective

The program is self-contained: no network, database or third-party library. At start-up it loads seven text files; after every change it saves them again. Global arrays hold the data and four classes hold the behaviour: `loginAccess`, `Register`, `Management` and `Rent`.

2.2 User classes

User	Portal	Main goals
Customer	Customer portal	Register, log in, rent one vehicle, view it, return it and see the cost
Owner	Owner portal	Register, log in, add up to 3 vehicles, view and remove them
Admin	Admin portal (admin / admin123)	Review customers, owners, vehicles (all, available, rented) and total revenue

2.3 Operating environment

Any OS with a C++11 compiler (tested with g++), a console, and write access to the folder it runs from.

2.4 Design and implementation constraints

- Fixed-size global arrays: at most 10 customers, 10 owners and 3 vehicles per owner.
- Text-file persistence with `|` as the field separator, so `|` is rejected in all text input.
- Console only; menus are numbered and the screen is "cleared" with blank lines.

2.5 Assumptions and dependencies

- One program instance uses the data files at a time.
- A vehicle's mileage is its odometer reading; rental cost is driven km times the vehicle's rate.
- Users are trusted to enter the true odometer reading on return.
- Currency is RS (rupees).

3. Requirements engineering

The requirements below were recovered from the working program, then checked by running it, so every requirement listed is one the code meets.

3.1 Process followed

Stage	What was done here
Elicitation	Read every menu, input prompt and message in the source; ran the program as each user class (reverse engineering and observation)
Analysis	Grouped behaviours by user class, removed duplicates, resolved conflicts (for example, "a customer may rent many vehicles" versus the code's one-at-a-time rule: the code's rule wins)
Specification	Gave each requirement a unique ID (UR, FR, NFR), a priority (Must, Should, Could) and the code that implements it
Validation	Ran scripted test sessions for each requirement (section 7)
Management	IDs are stable so a change request can cite them; the tables in section 4 are the traceability matrix

3.2 Stakeholders

Stakeholder	Interest
Customer	Easy registration, clear price, quick rent and return
Owner	Control over which vehicles are listed, correct payment for km driven
Admin	A complete, accurate view of users, fleet and revenue
Developer and instructor	A system that is testable, readable and traceable to requirements

3.3 User requirements

ID	As a … I want to …	Priority
UR-01	customer, register and log in with my email and password	Must
UR-02	customer, see which vehicles are free and their rate per km	Must
UR-03	customer, rent a free vehicle and later return it and see my cost	Must
UR-04	owner, register and log in	Must
UR-05	owner, add, view and remove my vehicles	Must
UR-06	admin, log in to a protected portal	Must
UR-07	admin, list and search customers and owners	Should
UR-08	admin, see all, available and rented vehicles	Should
UR-09	admin, see total revenue	Should
UR-10	any user, keep my data after the program closes	Must
UR-11	any user, get a clear message when I enter bad input	Should

4. Functional requirements

The system shall meet the 38 requirements below; the last column names the code that implements each, so the tables double as a traceability matrix.

4.1 Registration and login

ID	Requirement	Priority	Implemented in
FR-REG-01	The system shall register a customer or an owner from name, phone, email and password and give them the next sequential ID.	Must	`Register::registerCus`, `registerOwn`
FR-REG-02	The system shall reject a registration whose email already exists in the same user class.	Must	`Register::registerCus`, `registerOwn`
FR-REG-03	The system shall reject empty fields, fields containing `|`, emails not shaped like name@host.tld, and phones that are not 7 to 15 digits, `+` or `-`.	Should	`readField`, `readEmail`, `readPhone`
FR-REG-04	The system shall store at most 10 customers and 10 owners and say so when the limit is reached.	Must	`Register::registerCus`, `registerOwn`
FR-REG-05	The system shall use the lower-cased email as the username.	Must	`Register`, `loginAccess::getValues`
FR-AUT-01	The system shall grant customer access only when email and password match a registered customer, otherwise print ACCESS DENIED.	Must	`loginAccess::verifyCustomer`
FR-AUT-02	The system shall do the same for owners.	Must	`loginAccess::verifyOwner`
FR-AUT-03	The system shall grant admin access only for the built-in admin credentials.	Must	`loginAccess::verifyAdmin`

4.2 Owner and vehicle management

ID	Requirement	Priority	Implemented in
FR-OWN-01	An owner shall add a vehicle with name, identification number, mileage and rent per km; the system assigns the ID owner-sequence (for example 1-2).	Must	`Register::registerVehicle`
FR-OWN-02	The system shall allow at most 3 vehicles per owner.	Must	`Register::registerVehicle`
FR-OWN-03	The system shall keep vehicle identification numbers unique across all owners and re-ask on a duplicate.	Must	`vehNumberTaken`
FR-OWN-04	The system shall accept mileage of 0 or more and a rent per km above 0 only.	Must	`readDouble`
FR-OWN-05	An owner shall view their profile, vehicle count and each vehicle with its status.	Must	`ownPortal`, `printVehicle`
FR-OWN-06	An owner shall remove a vehicle after a Y/N confirmation.	Must	`ownPortal`
FR-OWN-07	The system shall refuse to remove a rented vehicle.	Must	`ownPortal`
FR-OWN-08	After a removal the system shall renumber the owner's remaining vehicle IDs and keep their rentals attached.	Should	`ownPortal`

4.3 Renting and returning

ID	Requirement	Priority	Implemented in
FR-RNT-01	A logged-in customer shall see their profile and whether they currently hold a vehicle.	Must	`cusPortal`

ID	Requirement	Priority	Implemented in
FR-RNT-02	The system shall list only vehicles that are not rented, with owner, ID, name, number, mileage and rate.	Must	`Rent::rentDetails`
FR-RNT-03	A customer shall rent a vehicle by identification number after a Y/N confirmation; a rented or unknown number is refused.	Must	`Rent::rentDetails`
FR-RNT-04	A customer shall hold at most one rented vehicle at a time.	Must	`isCusRenting` check
FR-RNT-05	A customer shall view only their own rented vehicle.	Must	`Rent::rentDetails`
FR-RNT-06	On return the customer shall enter the odometer reading; the system shall refuse a reading below the stored mileage.	Must	`Rent::rentDetails`
FR-RNT-07	The system shall compute cost = (new mileage - old mileage) x rent per km, show it, add it to revenue, store the new mileage, and free both vehicle and customer.	Must	`Rent::rentDetails`
FR-RNT-08	A customer shall be able to cancel a return and keep the vehicle rented.	Should	`Rent::rentDetails`

4.4 Administration

ID	Requirement	Priority	Implemented in
FR-ADM-01	The admin shall list all customers with ID, name, phone, email and rental status.	Should	`Management::cusManagement`
FR-ADM-02	The admin shall register a customer or an owner from the admin portal.	Could	`Management::cusManagement`, `ownManagement`

ID	Requirement	Priority	Implemented in
FR-ADM-03	The admin shall search a customer or an owner by email.	Should	`Management::cusManagement`, `ownManagement`
FR-ADM-04	The admin shall list all owners with their vehicle counts.	Should	`Management::ownManagement`
FR-ADM-05	The admin shall list all vehicles, only available vehicles, or only rented vehicles.	Should	`Management::listVehicles`
FR-ADM-06	The admin shall see total revenue in RS.	Should	`Management::transManagement`

4.5 Persistence and input handling

ID	Requirement	Priority	Implemented in
FR-PER-01	The system shall load customers, owners, vehicles, revenue and active rentals from text files at start-up.	Must	`loadAllData`
FR-PER-02	The system shall save after every change and on exit.	Must	`saveAllData`
FR-PER-03	After a restart, rented vehicles and customers holding them shall still show as rented.	Must	`loadAllData` (RENT_DETAILS.txt)
FR-PER-04	The system shall write report files of available and rented vehicles.	Could	`saveAllData`
FR-PER-05	The system shall skip malformed lines in data files instead of crashing.	Should	`loadAllData`
FR-INP-01	The system shall reject a non-numeric menu or number entry and ask again.	Should	`readInt`, `readDouble`

ID	Requirement	Priority	Implemented in
FR-INP-02	The system shall show a message for a menu choice outside the list and redisplay the menu.	Should	every menu function
FR-INP-03	Back and Exit choices shall return to the previous menu; the program ends only from main menu option 4, and shall also save and end if input is closed.	Must	menu loops, `readLine`

5. Non-functional, interface and data requirements

5.1 Non-functional requirements

ID	Category	Requirement
NFR-01	Usability	Every screen shall be a numbered menu with a prompt, and every message shall be in plain capitals text.
NFR-02	Reliability	Bad input (letters for numbers, empty or negative values) shall never crash the program or loop forever.
NFR-03	Data integrity	A vehicle shall never be rented to two customers, and a customer shall never hold two vehicles.
NFR-04	Persistence	Data shall survive a normal exit and a closed input stream.
NFR-05	Performance	Every menu action shall respond instantly for the maximum data set (10 customers, 10 owners, 30 vehicles).
NFR-06	Portability	The program shall build with any C++11 compiler using only the standard library.
NFR-07	Maintainability	Behaviour shall be grouped in four classes plus helper functions, and menus shall use loops, not recursion.

ID	Category	Requirement
NFR-08	Security (basic)	Portals shall require credentials; this version stores passwords as plain text (see section 8).
NFR-09	Capacity	Limits are 10 customers, 10 owners, 3 vehicles per owner, set by constants `MAX_USERS` and `MAX_VEH`.

5.2 External interfaces

- **User interface:** keyboard input and text output in a console. Each entry is read as a full line.
- **File interface:** plain text files in the working folder, read at start-up and written after each change.
- **Hardware and network:** none.

5.3 Data files

File	Content, one record per line (fields split by |)
`CUSTOMERS_DATA.txt`	id, name, phone, email, username, password, renting flag
`OWNERS_DATA.txt`	id, name, phone, email, username, password
`VEHICLES_DATA.txt`	owner index, vehicle ID, name, number, mileage, rate per km, rented flag
`RENT_DETAILS.txt`	customer index, owner index, vehicle index, then a readable copy of customer, owner and vehicle details
`REVENUE.txt`	total revenue
`AVAILABLE_VEH_DATA.txt`, `RENTED_VEH_DATA.txt`	report files for humans; not read back

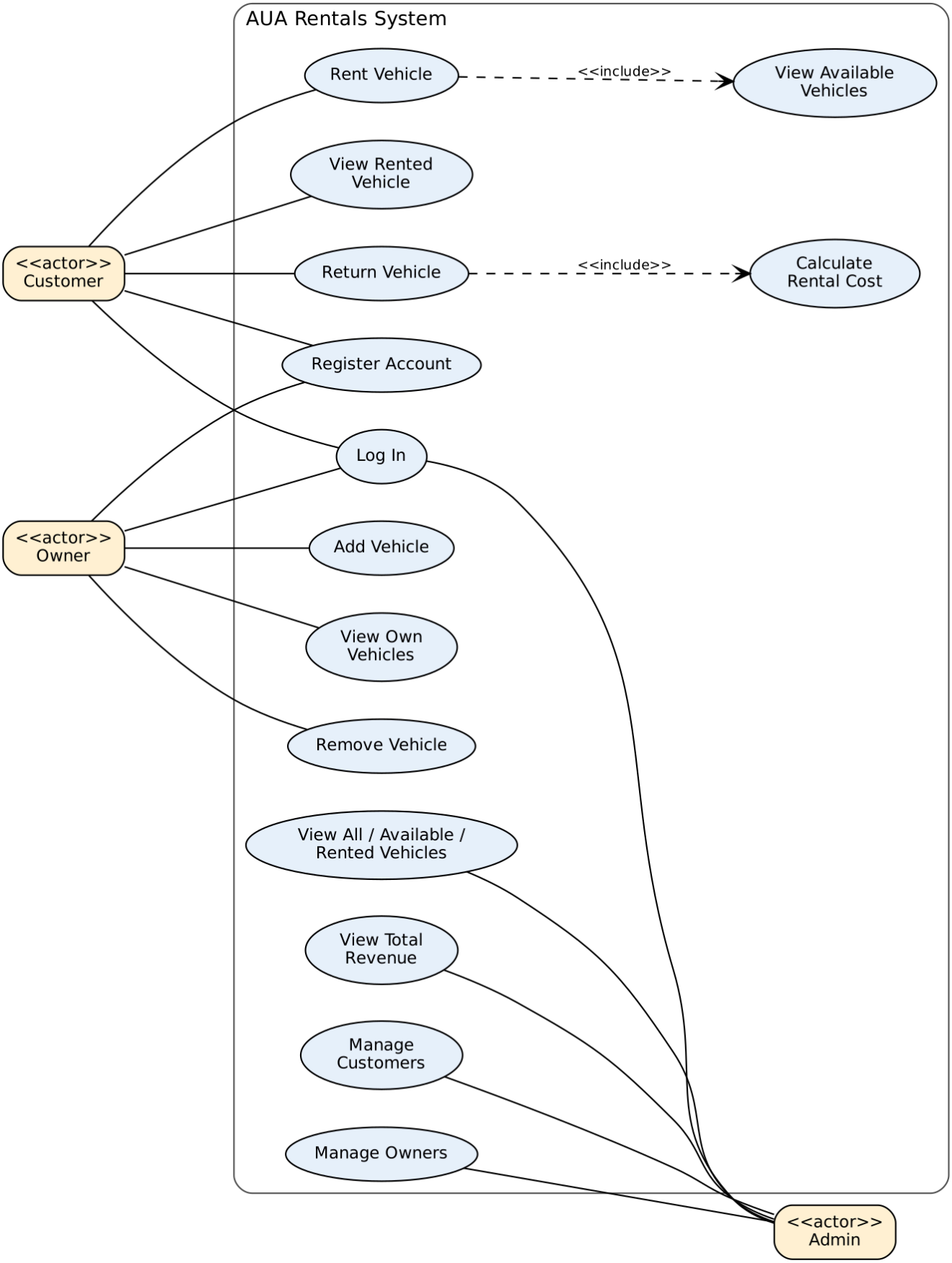
5.4 Business rules

- BR-01: Rental cost = (return mileage - pickup mileage) x rent per km, so no cost accrues without driven distance.
- BR-02: All rental cost is credited to total revenue and paid to the owner at return.
- BR-03: The return mileage becomes the vehicle's new pickup mileage.

6. UML models

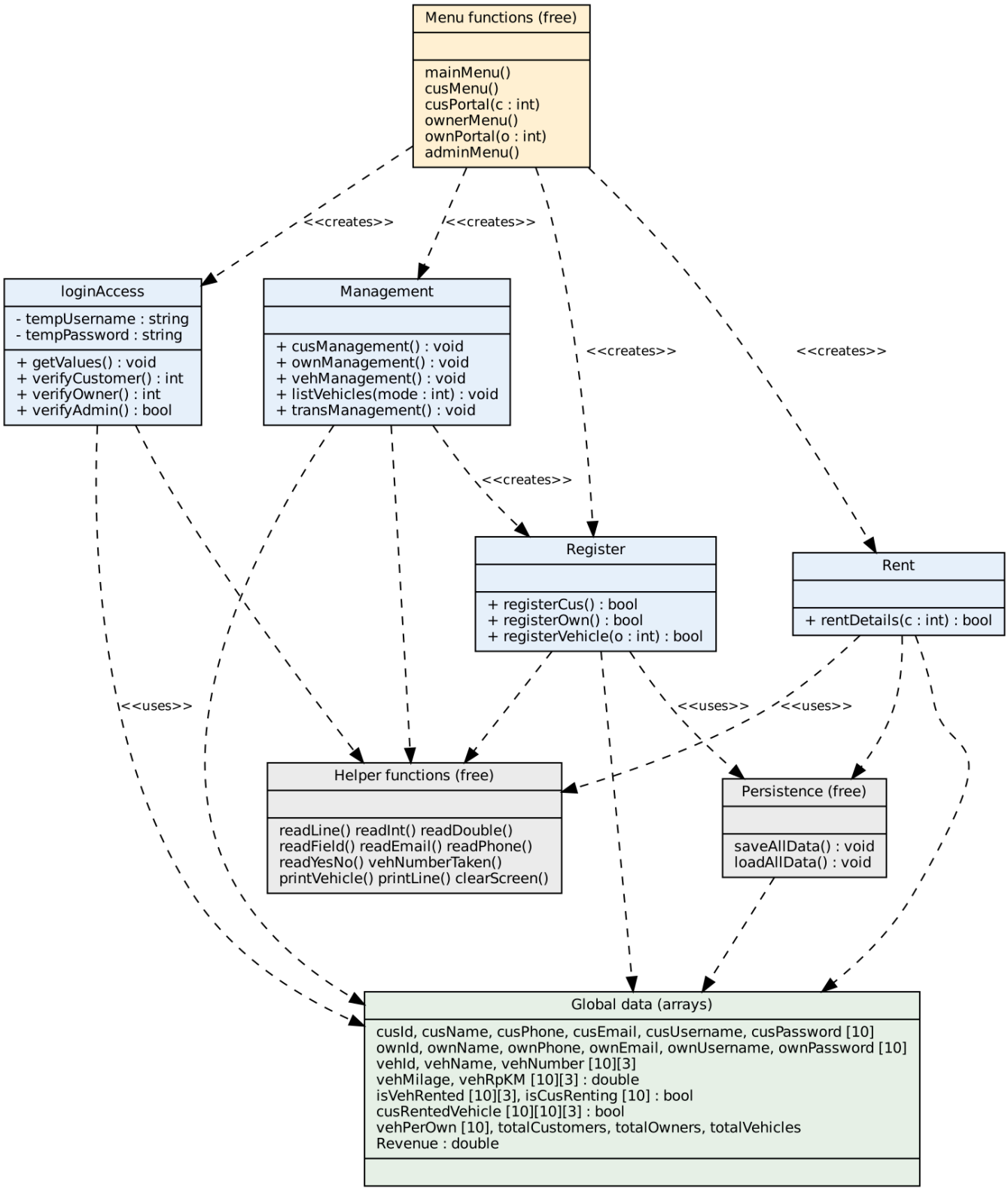
Eight diagrams describe the system from the user's view (use case), its structure (class), its behaviour (sequence, activity) and its life cycles (state).

6.1 Use case diagram


AUA Rentals System
Rent Vehicle
<<include>>
View Available Vehicles
View Rented Vehicle
<<actor>> Customer
Return Vehicle
<<include>>
Calculate Rental Cost
Register Account
Log In
<<actor>> Owner
Add Vehicle
View Own Vehicles
Remove Vehicle
View All / Available / Rented Vehicles
View Total Revenue
Manage Customers
Manage Owners
<<actor>> Admin

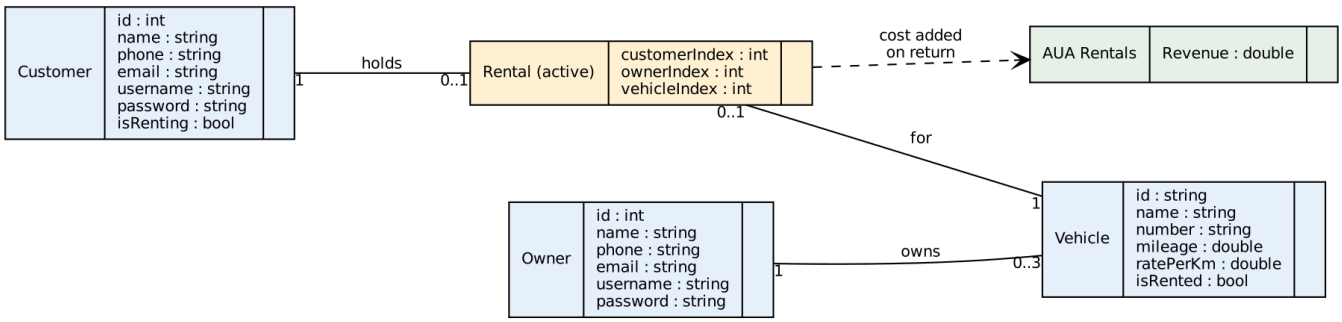
Customers and owners share Register and Log In; the admin owns the oversight use cases. Renting includes viewing available vehicles, and returning includes calculating the cost.

6.2 Class diagram: code structure



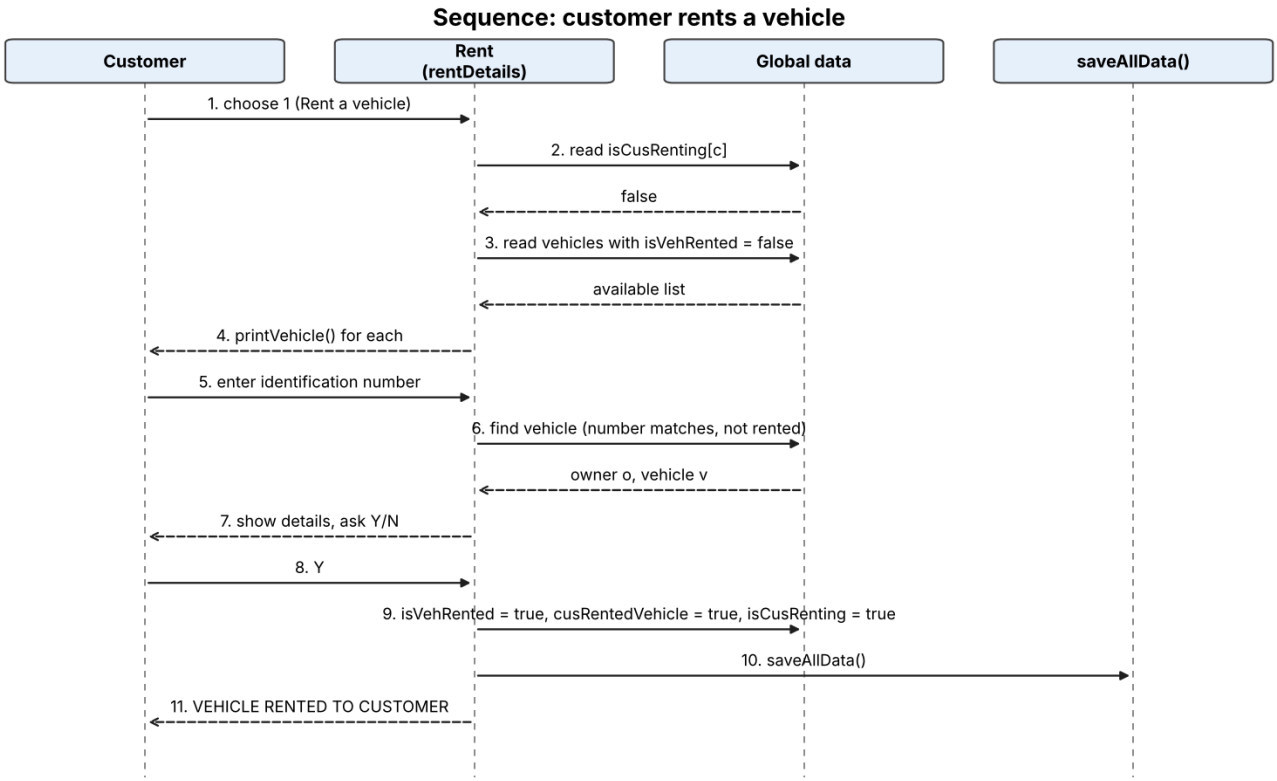
Menu functions create the four classes on demand; every class reads and writes the shared global arrays, and `Register` and `Rent` call `saveAllData()` after each change.

6.3 Class diagram: domain model

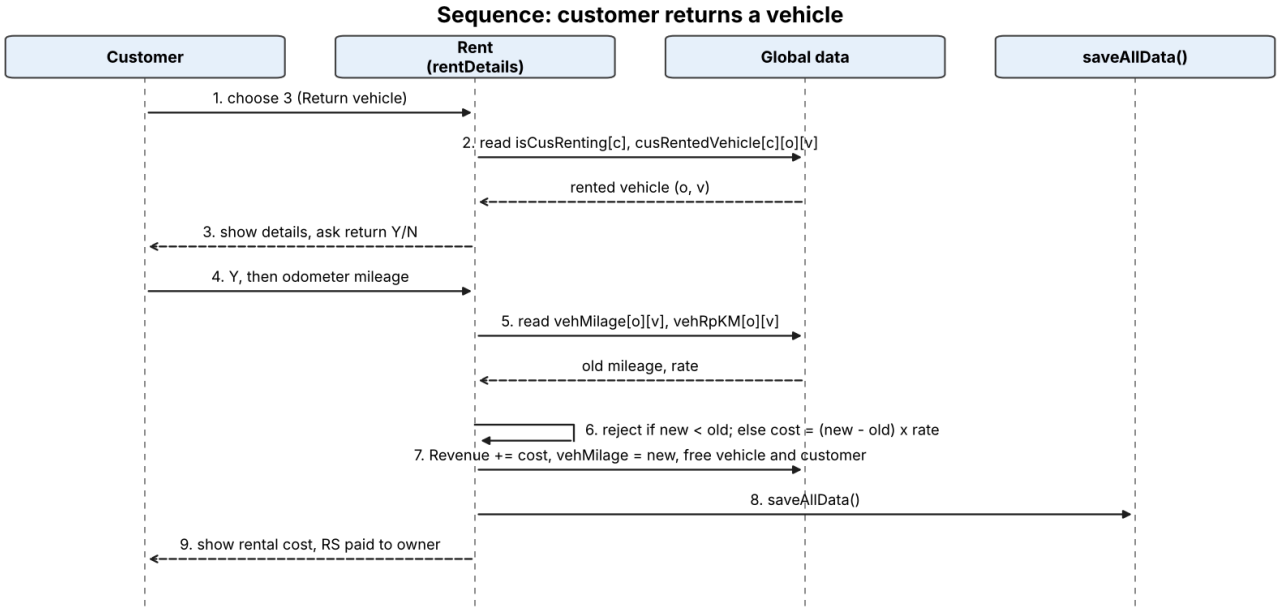


The 0..1 multiplicity between Customer and Rental is the one-vehicle-at-a-time rule (FR-RNT-04); the 0..3 on Vehicle is the per-owner limit (FR-OWN-02).

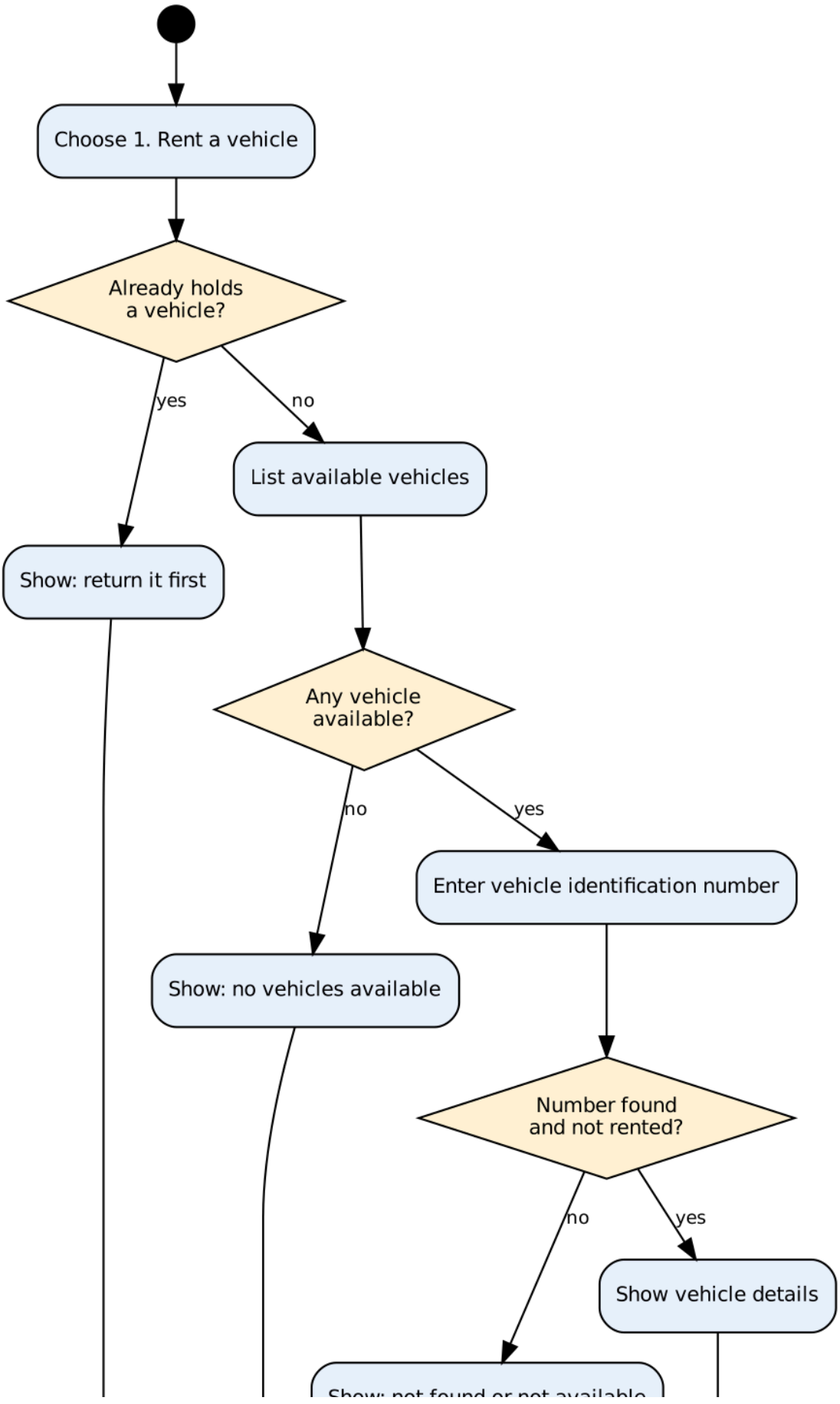
6.4 Sequence diagram: rent a vehicle

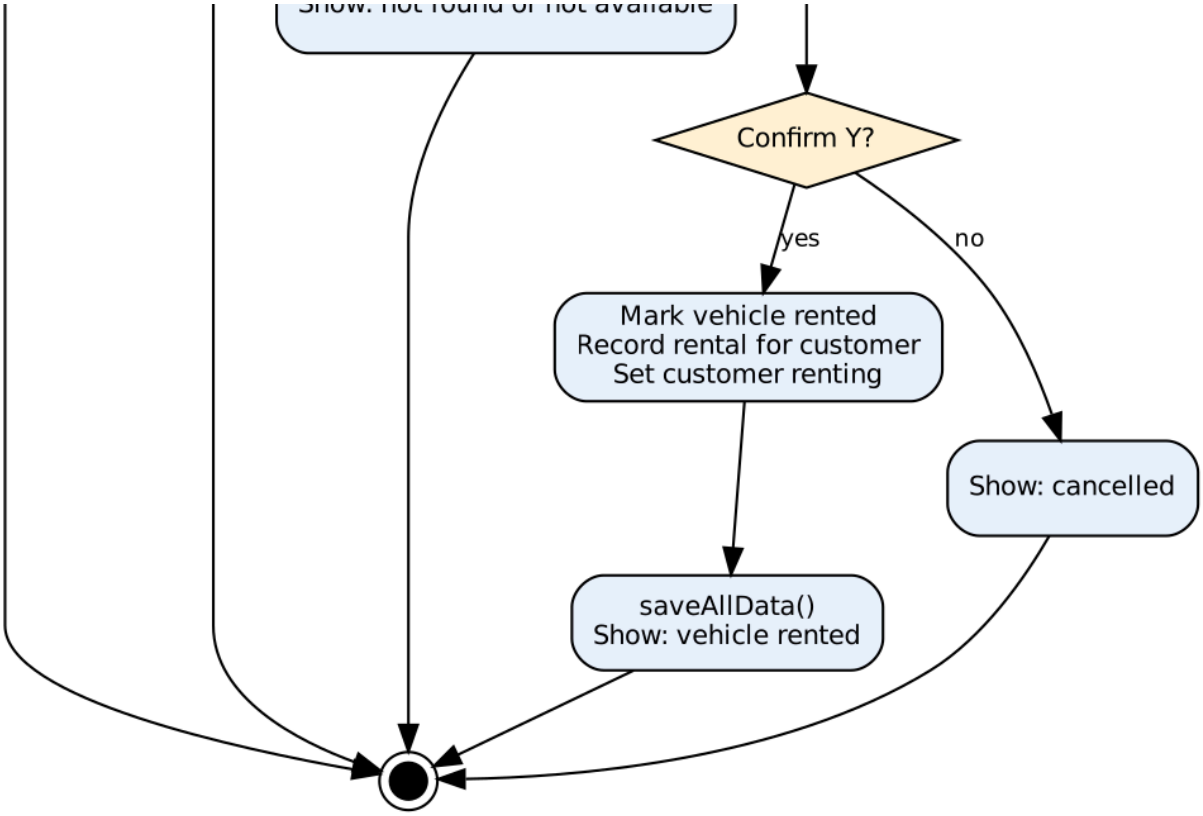


6.5 Sequence diagram: return a vehicle



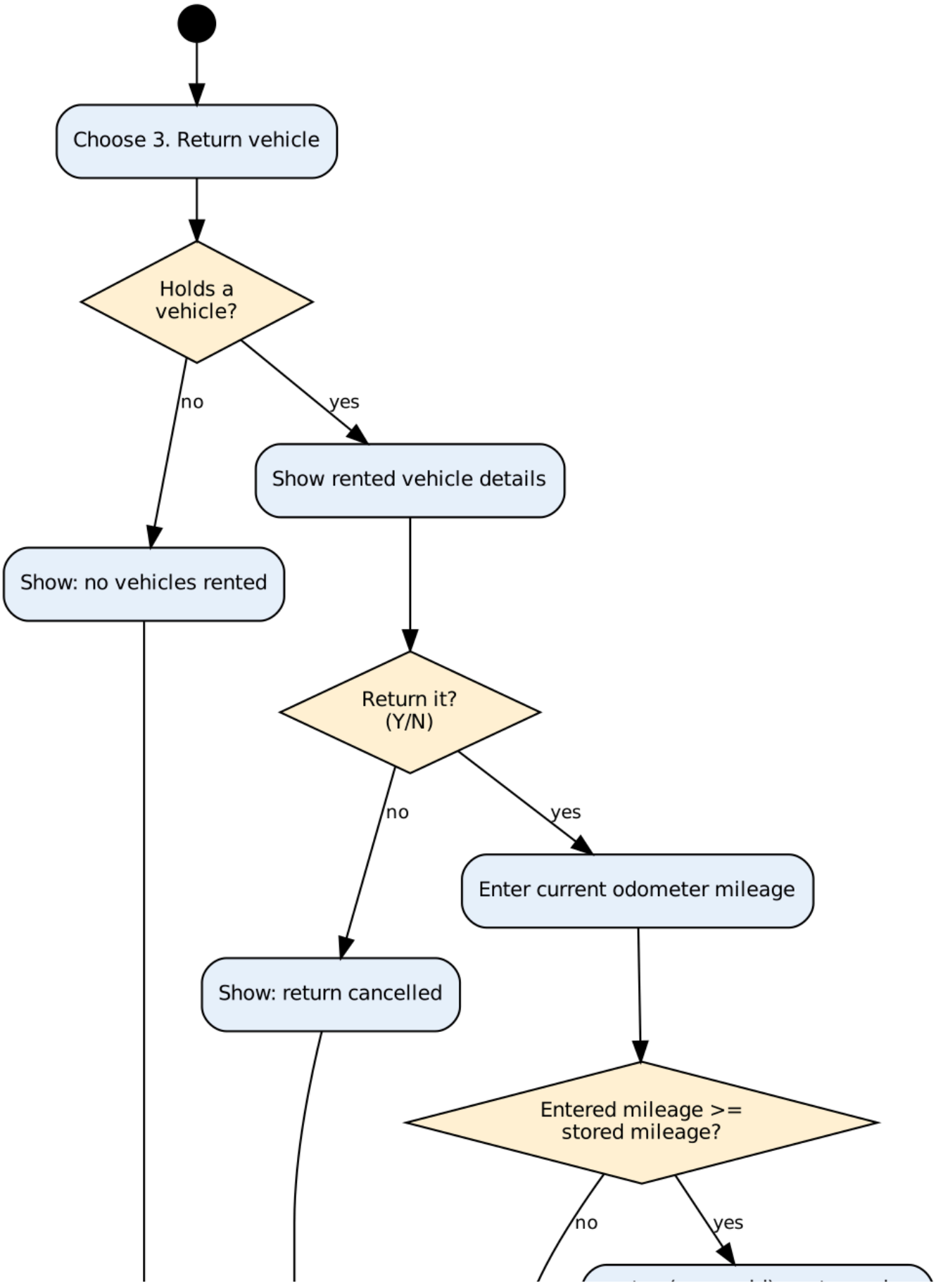
6.6 Activity diagram: rent

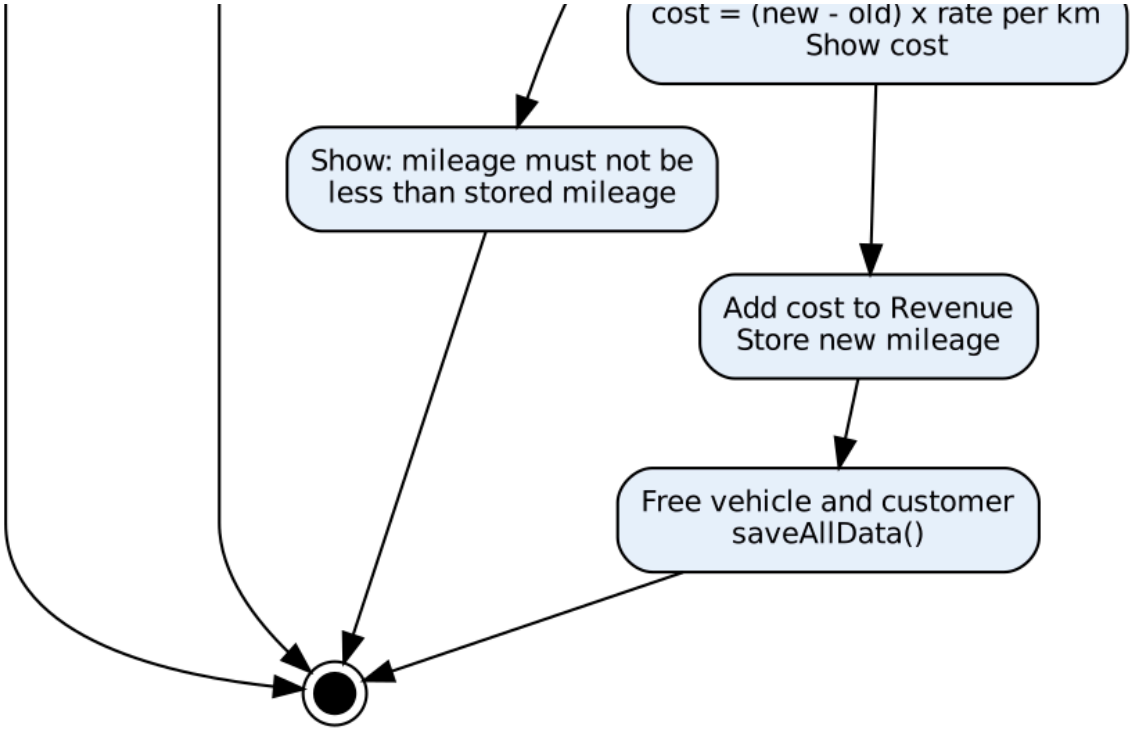

Choose 1. Rent a vehicle
Already holds a vehicle?
yes
no
Show: return it first
List available vehicles
Any vehicle available?
no
yes
Show: no vehicles available
Enter vehicle identification number
Number found and not rented?
no
yes
Show vehicle details



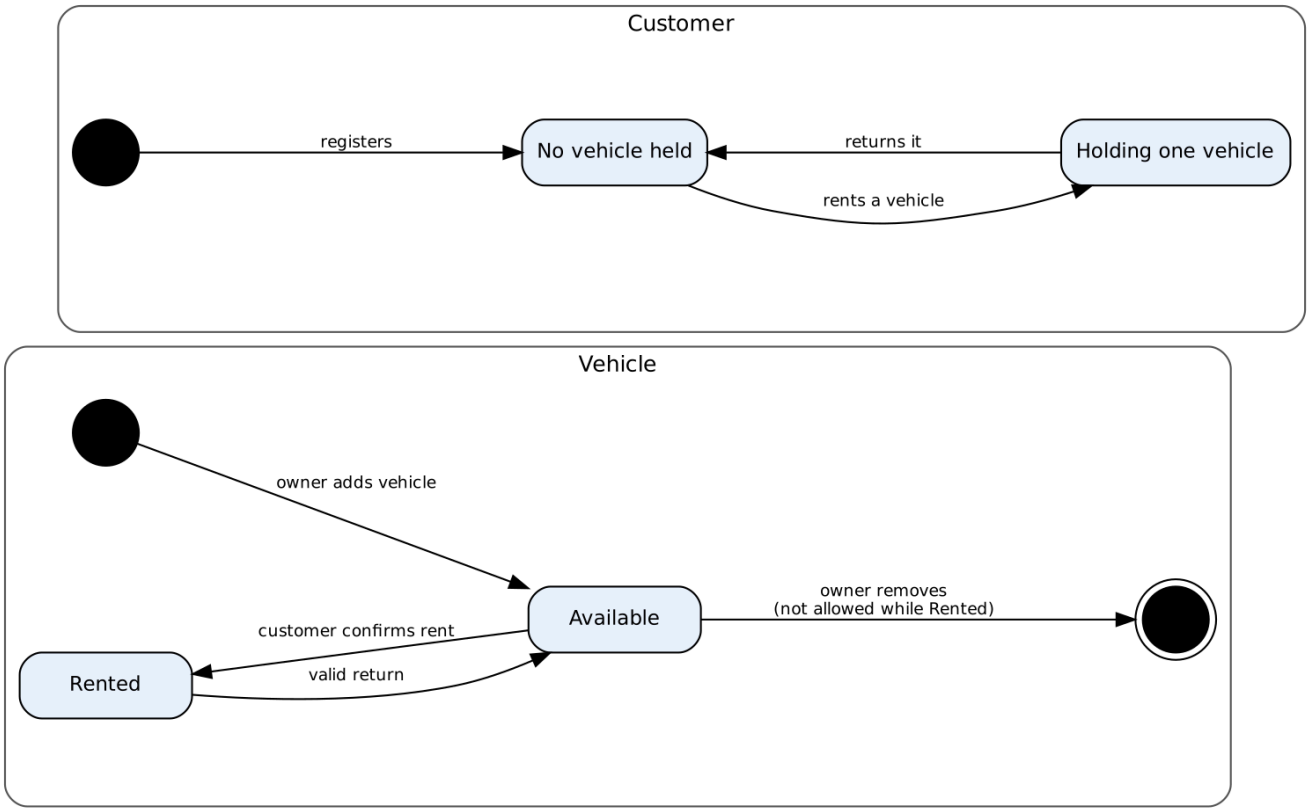
Four of the five exits are refusals (already renting, none available, number not found, cancelled); only a confirmed Y changes data.

6.7 Activity diagram: return


Choose 3. Return vehicle
Holds a vehicle?
no
yes
Show rented vehicle details
Show: no vehicles rented
Return it? (Y/N)
no
yes
Enter current odometer mileage
Show: return cancelled
Entered mileage >= stored mileage?
no
yes



6.8 State diagrams: vehicle and customer



A vehicle can only be removed from the Available state, and a customer moves between the two states only through rent and return.

7. Verification

The original program compiled, but reading it against the requirements exposed 15 defects; all are fixed in `car_rental.cpp`, and the fixed program passed every scripted test below.

7.1 Defects found in the original code

#	Defect in the original	Fix	Requirement
1	Menus called each other recursively, so choosing Exit unwound the stack and old menus reappeared; the program never ended cleanly	Every menu is a loop that returns to its caller	FR-INP-03, NFR-07
2	Typing letters at a number prompt put `cin` in a fail state and caused an endless loop	All input is read as a line and parsed; bad values are re-asked	FR-INP-01, NFR-02
3	Registering a customer set `vehPerOwn[customer index] = 0`, silently erasing an owner's vehicle count	Line removed	FR-OWN-05
4	The duplicate vehicle-number check looped over customers and `totalVehicles`, and compared the new vehicle with itself	New `vehNumberTaken()` checks all owners' real vehicles	FR-OWN-03
5	The "maximum vehicles" message opened the customer portal with an owner index	Returns to the owner portal	FR-OWN-02
6	A rented vehicle could still be typed in and rented again	Only vehicles with `isVehRented = false` match	FR-RNT-03, NFR-03
7	"View rented" and "Return" scanned every customer's rentals with wrong index limits, so one customer saw another's vehicle or missed their own	Scan only this customer's row, owners up to `totalOwners`, vehicles up to `vehPerOwn`	FR-RNT-05

#	Defect in the original	Fix	Requirement
8	The admin's rented-vehicle list printed the status NOT RENTED	One shared `printVehicle()` prints the true status	FR-ADM-05
9	Customer and owner search showed the menu first and printed "not found" afterwards	Result is printed before the menu returns	FR-ADM-03
10	Removing a vehicle with an unknown number carried on with index -1 (out-of-range access)	Returns to the menu when not found	FR-OWN-06
11	A rented vehicle could be removed, leaving a rental pointing at nothing; after a removal two vehicles could share an ID	Removal blocked while rented; IDs and rental flags are shifted and renumbered	FR-OWN-07, FR-OWN-08
12	Invalid choices and Exit in the owner portal and rent menu jumped to the wrong menu	Each menu redisplays itself or returns to its caller	FR-INP-02
13	Mileage and rate were `float` and accepted negative or zero values; names with `|` or empty fields broke the data files	`double`, range checks, field validation	FR-OWN-04, FR-REG-03
14	Two customers with the same name could not both register	Uniqueness is by email only	FR-REG-02
15	"ALL DATA SAVED" printed after every change; the names `isCusAvailable` and `isVehAvailable` actually meant "rented"	Silent saves; renamed to `isCusRenting` and `isVehRented`	NFR-01

7.2 Test results

The fixed program was built with `g++ -std=c++11 -Wall -Wextra` (no warnings) and driven by scripted console sessions, with the program restarted between sessions to prove persistence.

Test	Expected	Result
Register owner, log in with a mixed-case email	Access granted	Pass
Add 3 vehicles	IDs 1-1, 1-2, 1-3	Pass
Add a duplicate vehicle number	Re-asked	Pass
Enter letters, then -5, as mileage	Re-asked until valid	Pass
Add a 4th vehicle	Refused, back to owner portal	Pass
Type letters at a menu	Re-asked, no loop	Pass
Register a customer twice with one email	Second refused	Pass
Rent a free vehicle, then try to rent another	First succeeds, second refused	Pass
Return at a lower mileage than stored	Refused	Pass
Return at 1100 from 1000 at 20 RS per km	Cost 2000.00, revenue 2000.00	Pass
Owner removes a rented vehicle	Refused	Pass
Owner removes a free vehicle	Deleted; the rented vehicle becomes 1-1 and stays rented	Pass
Restart after renting	Vehicle and customer still show rented	Pass
Admin wrong password, then correct	Denied, then granted	Pass
Admin lists all, available, rented vehicles; revenue	Correct statuses and amounts	Pass
Choose Exit	Data saved, process ends with status 0	Pass
Input stream closed mid-menu	Data saved, process ends	Pass

These are manual scripted runs, not an automated unit-test suite; adding one is listed in section 8.

8. Known limitations and future work

The program meets its stated scope; the items below are gaps a reviewer should know about, with a suggested next step for each.

Limitation	Why it matters	Suggested next step
Passwords are stored in plain text and the admin login is fixed in the source	Anyone who can open the data files can read every password	Store salted hashes; read the admin credentials from a file
Arrays are fixed at 10 customers, 10 owners, 3 vehicles per owner	The system cannot grow	Replace arrays with structs in `std::vector`, keeping the same menus
Only the running revenue total is kept, with no rental history or per-owner earnings	No receipts or owner statements	Append each completed rental to a history file
One rental at a time and no booking dates	Cannot reserve ahead or rent several vehicles	Add start and end dates and a rental ID
The returned odometer reading is trusted	A customer could under-report distance	Owner confirms the reading before cost is charged
No way to change a password or delete a user	Mistaken accounts stay forever	Add profile edit and admin delete with rental checks
Two copies of the program on one folder would overwrite each other's data	Data loss	Lock file, or a real database
Tests are manual scripted sessions	Regressions can slip in	Add an automated test script that pipes input and compares output